



مرئي برمجة

مقدمة في برمجة واجهة المستخدم الرسومية

مصباح ج. موسى

إياس الشيخ خليل

وصف الدورة

- برمجة التطبيقات باستخدام واجهات المستخدم الرسومية.

- تتضمن الموضوعات عناصر التحكم والأحداث والموارد وربط البيانات والرسوم المتحركة والأنماط.

- سوف يتعلم الطلاب عملية إنشاء تطبيقات سطح المكتب من البداية إلى النهاية باستخدام Java.

- ستتعلم كيفية الاستفادة من معرفتك ببرمجة Java واستكشاف مكتبات JavaFX لتصميم واجهات رسومية ورسوم متحركة بسيطة وألعاب صغيرة.

- سيتعلم الطلاب كيفية ربط تطبيقات Java بقواعد البيانات.

نتائج تعلم الدورة

1. بناء وتعزيز مهارات البرمجة وتصميم البرمجيات.

2. فهم وتطبيق مبادئ تصميم واجهة المستخدم الأساسية.

3. فهم وتعلم كيفية استخدام واجهات برمجة التطبيقات والأدوات المتاحة لبناء وتصميم: الرسوم المتحركة والألعاب وواجهات المستخدم.

4. تعلم كيفية ربط البرامج بقواعد البيانات.

5. بناء تطبيق متكامل يحتوي على واجهات مستخدم ويتصل بقاعدة بيانات.

الخطوط العريضة

- البرمجة المرئية

- JavaFX مقابل Swing و AWT.

- ما هو JavaFX؟

- مميزات JavaFX.

- تاريخ JavaFX.

- بنية تطبيق JavaFX.

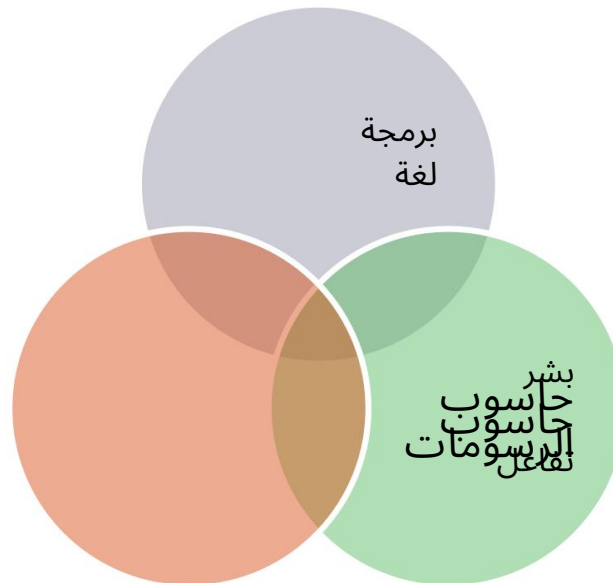
- دورة حياة تطبيق JavaFX.

- أول تطبيق JavaFX

البرمجة المرئية

• واجهة المستخدم الرسومية (GUI) هي واجهة رسومية تمثل تمثيلًا مرئيًا للاتصالات يتم تقديمه للمستخدم للتفاعل بسهولة مع الجهاز.

• إنها واجهة المستخدم المشتركة التي تتضمن التمثيل الرسومي مثل الأزرار والأيقونات، ويمكن إجراء الاتصال عن طريق التفاعل مع هذه الأيقونات بدلاً من الاتصال المعتاد القائم على النص أو الأوامر.



البرمجة المرئية

أطر اللغات



Windows Forms

WPF



AWT وSWING مقابل JAVA FX

- تم استبدال Swing و AWT بمنصة JavaFX للتطوير تطبيقات الإنترنت الغنية.

- عندما تم تقديم Java، تم تجميع فئات واجهة المستخدم الرسومية في مكتبة المعروفة باسم مجموعة أدوات Windows المجردة (AWT).

يُعد AWT مناسبًا لتطوير واجهات مستخدم رسومية بسيطة، ولكنه ليس مناسبًا لتطوير مشاريع واجهة مستخدم رسومية شاملة. كما أن AWT عرضة لأخطاء خاصة بالمنصة.

- تم استبدال مكونات واجهة مستخدم AWT بمكتبة أكثر قوة وتنوعًا ومرونة تُعرف باسم مكونات Swing .

- يتم رسم مكونات التآرجح مباشرة على القماش باستخدام كود Java.

- تعتمد مكونات التآرجح بشكل أقل على منصة الهدف وتستخدم قدرًا أقل من مورد واجهة المستخدم الرسومية الأصلي.

- مع إصدار Java 8، تم استبدال Swing بواجهة مستخدم رسومية جديدة تمامًا منصة تُعرف باسم JavaFX.

ما هو JAVAFX؟

JavaFX هي مكتبة جافا تُستخدم لتطوير تطبيقات سطح المكتب، بالإضافة إلى تطبيقات الإنترنت الغنية (RIA). يمكن تشغيل التطبيقات المُدمجة في JavaFX على منصات متعددة، بما في ذلك الويب، والهواتف المحمولة، وأجهزة الكمبيوتر المكتبية.

• صُمم JavaFX ليحل محل Swing في تطبيقات Java لإطار عمل لواجهة المستخدم الرسومية. ومع ذلك، فهو يوفر وظائف أكثر من Swing.

مثل Swing، يوفر JavaFX أيضاً مكوناته الخاصة ولا يعتمد على نظام التشغيل. يتميز بخفة وزنه وتسريعه عبر الأجهزة. يدعم أنظمة تشغيل متنوعة، بما في ذلك Linux وWindows وOS. caM.

مميزات JAVAFX

- مكتوب بلغة جافا. هذه التقنية الغنية متوفرة ضمن مجموعة أدوات جافا الإصدار 8 أو أعلى.
- تدعم هذه التقنية تصميم CSS.
- إنه يوفر محرك JavaScript.
- يمكننا إضافة كائن رسومي ثنائي/ثلاثي الأبعاد إلى تطبيقاتنا.
- يدعم الصورة وتنسيق الوسائط (mp3، aiff) والفيديو (flv).
- يحتوي على WebView بحيث يمكن إضافة مكونات HTML أيضًا باستخدام JavaFX.
- نظرًا لأنه مبني على الجزء العلوي من التآرجح، فيمكننا استخدام مميزات التآرجح أيضًا في جافا اف اكس.
- إنه يدعم استخدام XML للحصول على هياكل بيانات منفصلة يتم صيانتها بشكل مختلف عن كود التطبيق.
- يمكنك تجنب متاعب أخطاء HTML المعتمدة على المتصفح ومشاكل الأداء بفضل ثراء تقنية JavaFX.

– JavaFX الهندسة المعمارية

JavaFX API

Scene Graph

Quantum Toolkit

Prism

Glass

Web view

Media

**Win32
GTK**

**OpenGL
D3D**

Web kit

**G
Streams**

JDK API and Tools

Java Virtual Machine

– JAVA FX الهندسة المعمارية

• واجهة برمجة تطبيقات JavaFX العامة

• تحتوي الطبقة العليا من بنية JavaFX على واجهة برمجة تطبيقات JavaFX العامة توفر جميع الفئات الضرورية المسؤولة عن تنفيذ تطبيق JavaFX كامل الميزات.

• رسم بياني للمشهد

•

إنها نقطة البداية لبناء تطبيق JavaFX.

إنها شجرة هرمية من العقد تمثل جميع العناصر المرئية لواجهة المستخدم. كما أنها قادرة على معالجة الأحداث. بشكل عام، يُمكن تعريف مخطط المشهد كمجموعة من العقد.

لكل عقدة مُعرّف ونمط وحجم مُنفصل. لا يُمكن لكل عقدة في رسم بياني للمشهد أن تحتوي إلا على عقدة رئيسية واحدة، ولا تحتوي على أي عقد فرعية أو أكثر.

– JavaFX الهندسة المعمارية

• محرك الرسومات

يوفر محرك رسومات **JavaFX** دعمًا رسوميًا لمخطط المشهد. وهو يدعم بشكل أساسي الرسومات ثنائية الأبعاد وثلاثية الأبعاد. كما يوفر إمكانية العرض البرمجي عندما لا تدعم وحدة معالجة الرسومات الموجودة على النظام العرض المُسرَّع بالأجهزة.

• خطي الأنابيب المتسارعين للرسومات في JavaFX هما:

1. المنشور

• يمكن اعتبار المنشور بمثابة أجهزة عالية الأداء-

خط أنابيب رسومات مُسرَّع. لديه القدرة على عرض الرسومات ثنائية وثلاثية الأبعاد. يُطبَّق Prism طرقًا مختلفة لعرض الرسومات على منصات مختلفة.

• **DirectX 9** على نظامي التشغيل Vista أو Windows XP

• **DirectX 11** على نظام التشغيل Windows 7

• **OpenGL** على Linux وMac والمضمنة

• **Java 2D** عندما لا يكون تسريع الأجهزة ممكنًا

- JAVA FX الهندسة المعمارية

2. مجموعة أدوات الكم

• يتم استخدام مجموعة الأدوات الكمومية لربط المنشور والزجاج مجموعة أدوات النوافذ معًا وجعلها متاحة للطبقات المذكورة أعلاه في المكس.

• مجموعة أدوات تركيب النوافذ الزجاجية

- إنه موجود في أدنى مستوى من مجموعة رسومات JavaFX. يمكن اعتباره أساسًا بمثابة طبقة تعتمد على المنصة والتي تعمل كواجهة بين منصة JavaFX ونظام التشغيل الأصلي.
- وهو المسؤول عن توفير خدمات نظام التشغيل مثل إدارة النوافذ والمؤقتات وقوائم الأحداث والأسطح.

- JAVA FX الهندسة المعمارية

• عرض الويب

يمكننا أيضًا تضمين محتوى HTML في رسم بياني لمشهد JavaFX لهذا الغرض، يستخدم JavaFX مكونًا يُسمى "عرض الويب".
يستخدم "عرض الويب" Web Kit، وهو متصفح داخلي مفتوح المصدر، ويُمكنه عرض CSS و DOM و HTML5 و JavaScript و SVG

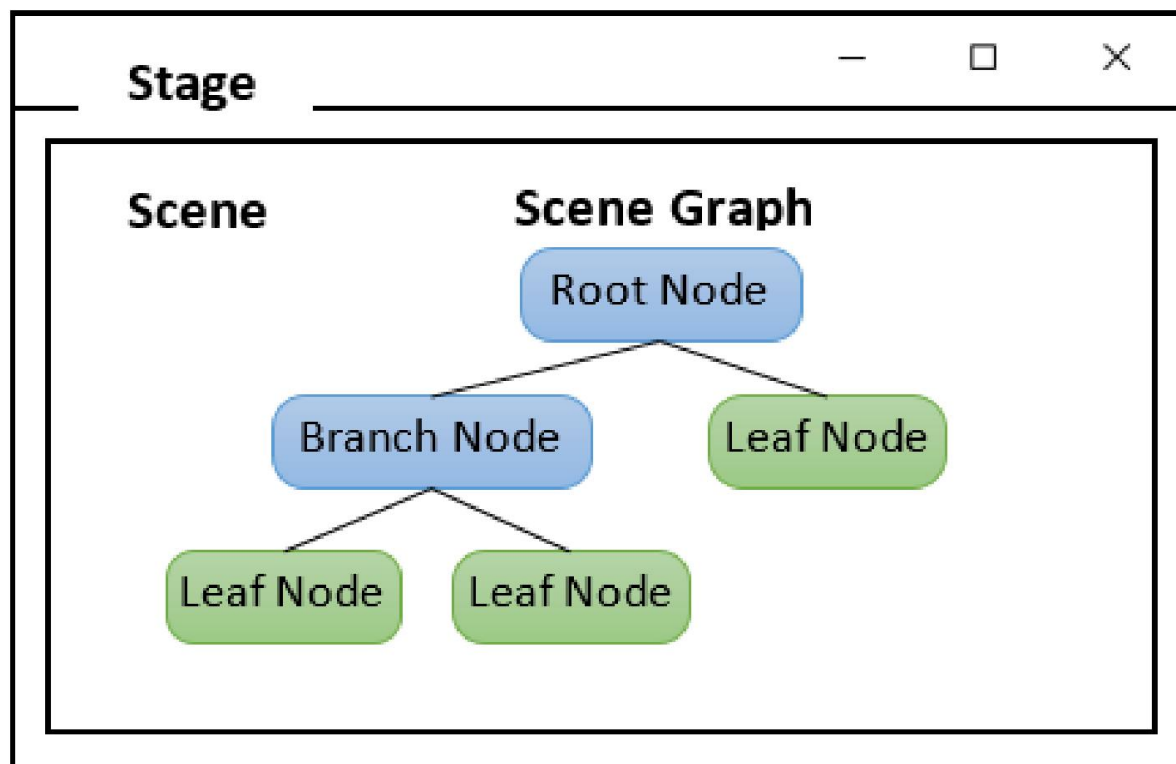
• محرك الوسائط

• باستخدام محرك الوسائط، يمكن لتطبيق JavaFX تشغيل ملفات الوسائط الصوتية والفيديو.

يعتمد محرك وسائط JavaFX على محرك مفتوح المصدر يُسمى G Streamer. تحتوي الحزمة javafx.scene.media على جميع الفئات والواجهات التي تُمكن تطبيقات JavaFX من توفير وظائف الوسائط.

بنية تطبيق JavaFX

- بشكل عام، سيحتوي تطبيق JavaFX على ثلاثة مكونات رئيسية وهي Nodes و Scene و Stage كما هو موضح في الرسم التخطيطي التالي.



بنية تطبيق JAVAFX

• منصة

تحتوي **المرحلة** (النافذة) على جميع عناصر تطبيق JavaFX. تُمثّل المرحلة من الحزمة `javafx.stage` لعرض محتويات المرحلة، يجب استدعاء دالة `show()`.

• مشهد

يحتوي **المشهد** فعليًا على جميع المحتويات المادية (العُقد) لتطبيق JavaFX. `javafx.scene.Scene` جميع الطرق للتعامل مع كائن المشهد. إنشاء المشهد ضروري لعرض محتوياته .

بنية تطبيق JavaFX

- رسم بياني للمشهد

- الرسم البياني للمشهد عبارة عن بنية بيانات مشابهة للشجرة، في التطبيقات الرسومية الحديثة، هي عبارة عن مجموعة من العقد. في تطبيقات JavaFX، تحتوي فئة `javafx.scene.scene` على جميع محتويات رسم المشهد.

- أثناء إنشاء مشهد، من الضروري تحديد الجذر

العقدة.

- في إحدى الحالات، تتم إضافة كائن المشهد إلى مرحلة واحدة فقط.

بنية تطبيق JavaFX

• العقد

• العقدة هي كائن بدائي مرئي/رسومي لتطبيق JavaFX.

• كائنات هندسية ثنائية وثلاثية الأبعاد (دائرة، كرة، إلخ)، واجهة المستخدم تعتبر كائنات التحكم (الزر، مربع الاختيار، وما إلى ذلك)، وكائنات الحاوية/التخطيط (جزء الحدود، جزء الشبكة، وما إلى ذلك)، وكائنات عناصر الوسائط (كائنات الصوت والفيديو والصورة، وما إلى ذلك) (بعض الأمثلة) للعقد في JavaFX.

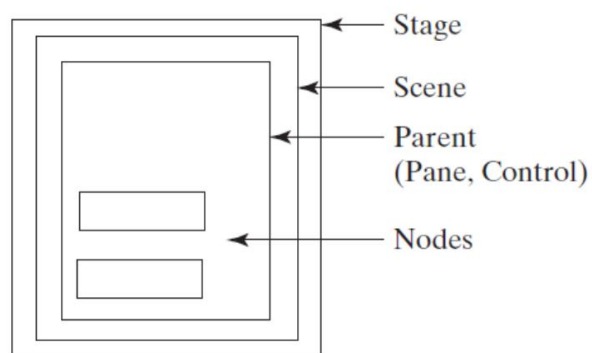
• كل عقدة في الرسم البياني للمشهد لها والد واحد، و العقدة التي لا تحتوي على أي آباء تُعرف باسم العقدة الجذرية.

• كل عقدة رئيسية لديها طفل واحد أو أكثر، و تُسمى العقدة التي لا تحتوي على أطفال بالعقدة الورقية، وتُسمى العقدة التي تحتوي على أطفال بالعقدة الفرعية.

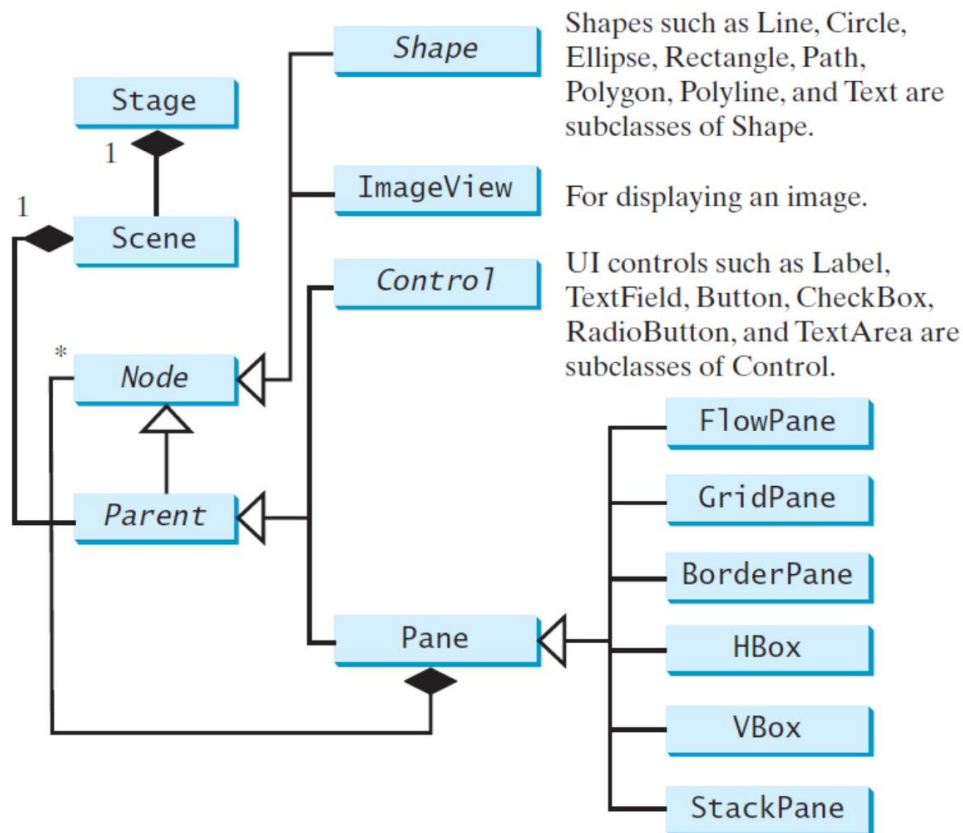
بنية تطبيق JAVA FX

ماي جافا اف اكس

عرض توضيحي متعدد المراحل



(a)



(b)

بنية تطبيق JavaFX

• فيما يلي أنواع مختلفة من العقد الجذرية في تطبيق JavaFX.

• المجموعة -عقدة المجموعة هي عقدة جماعية تحتوي على قائمة من العقد الفرعية. عند عرض عقدة المجموعة، تُعرض جميع العقد الفرعية بالترتيب. أي تحويل أو حالة تأثير تُطبق على المجموعة ستُطبق على جميع العقد الفرعية.

• المنطقة □ هي الفئة الأساسية لجميع عناصر التحكم في واجهة المستخدم المستخدمة المستندة إلى JavaFX Node، مثل الرسم البياني والجزء والتحكم.

□ WebView • تقوم هذه العقدة بإدارة محرك الويب وعرض محتوياته.

• عقدة الورقة -العقدة التي لا تحتوي على عقد فرعية تُعرف باسم

عقدة ورقية. على سبيل المثال، المستطيل، والقطع الناقص، والمربع، وعرض الصورة، وعرض الوسائط هي أمثلة على عقد ورقية.

دورة حياة تطبيق JavaFX

• تحتوي فئة تطبيق JavaFX على ثلاث طرق لدورة الحياة، وهي □

□ start() • طريقة نقطة الدخول التي سيتم كتابة كود رسومات JavaFX فيها.

□ stop() • طريقة فارغة يمكن تجاوزها، هنا يمكنك كتابة المنطق لإيقاف التطبيق.

□ init() طريقة فارغة يمكن تجاوزها، ولكن لا يمكنك إنشاء مرحلة أو مشهد في هذه الطريقة.

دورة حياة تطبيق JavaFX

- بالإضافة إلى ذلك، فإنه يوفر طريقة ثابتة تسمى `launch()` لتشغيل تطبيق JavaFX.
- عند تشغيل تطبيق JavaFX، سيتم تنفيذ الإجراءات التالية (بنفس الترتيب).
 - يتم إنشاء مثيل لفئة التطبيق.
 - يتم استدعاء طريقة `Init()`.
 - يتم استدعاء طريقة `start()`.
 - ينتظر المشغل انتهاء التطبيق ثم يستدعي طريقة `stop()`.

دورة حياة تطبيق JAVAFX

• لا تكون هناك حاجة للطريقة الرئيسية إذا قمت بتشغيل البرنامج من سطر الأوامر.

- قد يكون من الضروري تشغيل برنامج JavaFX من IDE مع دعم JavaFX محدود.
- عند تشغيل تطبيق JavaFX بدون طريقة رئيسية، يستدعي VM تلقائيًا طريقة التشغيل لتشغيل التطبيق.

دورة حياة تطبيق JavaFX

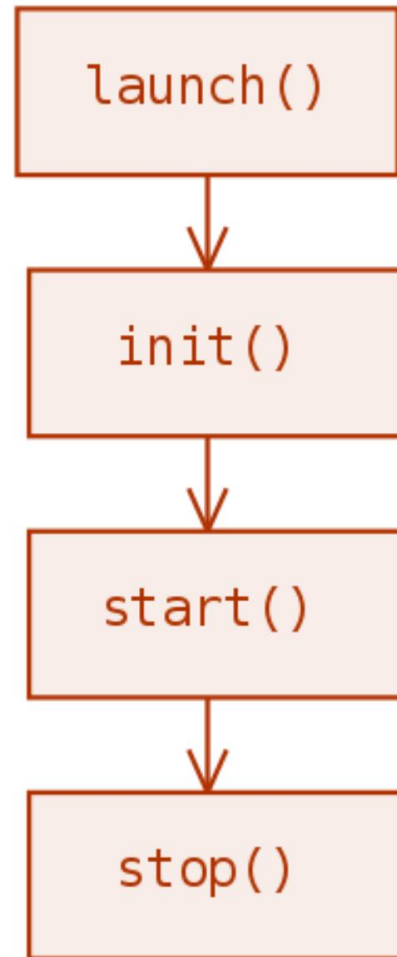
• إنهاء تطبيق JavaFX

• عند إغلاق النافذة الأخيرة للتطبيق، يتم إنهاء تطبيق JavaFX ضمناً.

• يمكنك إيقاف تشغيل هذا السلوك عن طريق تجاوز القيمة المنطقية "False" إلى الطريقة الثابتة `setImplicitExit()` (يجب استدعاؤها من سياق ثابت).

• يمكنك إنهاء تطبيق JavaFX صراحةً باستخدام طريقتي `Platform.exit()` أو `System.exit(int)`.

دورة حياة تطبيق JAVAFX



أول تطبيق JAVAFX

الخطوة 1: توسيع `javafx.application.Application` وتجاوز `start()`

• كما درسنا سابقاً أن طريقة `start()` هي نقطة البداية لبناء تطبيق `JavaFX`، لذلك نحتاج أولاً إلى تجاوز طريقة `start` الخاصة بفئة `javafx.application.Application`.

يتم تمرير كائن الفئة `javafx.stage.Stage` إلى `start()` لذلك قم باستيراد هذه الفئة ومرر كائنها إلى طريقة البدء. يجب استيراد `javafx.application.Application` لتجاوز طريقة البدء.

```
package application;
import javafx.application.Application;
import javafx.stage.Stage;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub

    }

}
```

أول تطبيق JavaFX

• الخطوة 2: تشغيل التطبيق

- سوف نستدعي طريقة `launch()` ونمرر سطر الأوامر الحجج (`args`) في الطريقة الرئيسية.

```
/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    launch(args);
}
```

أول تطبيق JAVAFX

• الخطوة 3: إنشاء زر

يمكن إنشاء زر بإنشاء مثيل لفئة `javafx.scene.control.Button`. يجب استيراد هذه الفئة إلى الكود. مرر نص تسمية الزر في مُنشئ فئة الزر.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");

    }

}
```

أول تطبيق JavaFX

• الخطوة 4: إنشاء تخطيط وإضافة زر إليه

يوفر **JavaFX** عددًا من التخطيطات. نحتاج إلى تنفيذ أحدها لعرض عناصر واجهة المستخدم بشكل صحيح. يوجد هذا التخطيط في أعلى مخطط المشهد، ويمكن اعتباره عقدة جذرية.

يجب إضافة جميع العقد الأخرى (الأزرار والنصوص وما إلى ذلك) إلى هذا التخطيط.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);

    }
}
```

أول تطبيق JAVAFX

الخطوة 5: إنشاء مشهد

يجب إضافة التخطيط إلى مشهد. يبقى المشهد في المستوى الأعلى في هيكل التطبيق. يمكن إنشاؤه بإنشاء مثيل للفئة `javafx.scene.Scene`. نحتاج إلى تمرير كائن التخطيط إلى مُنشئ فئة المشهد.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);
        Scene scene=new Scene( parent:root);

    }

}
```

أول تطبيق JAVAFX

الخطوة 6: تحضير المسرح

يوفر **صف** `javafx.stage.Stage` بعض الدوال المهمة التي يجب استدعاؤها لتعيين بعض سمات المرحلة. يمكننا تعيين عنوان المرحلة. كما نحتاج إلى استدعاء دالة `show()` والتي بدونها لن يتم عرض المرحلة.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);
        Scene scene=new Scene( parent:root);
        primaryStage.setScene( value: scene);
        primaryStage.setTitle( value: "First JavaFX Application");
        primaryStage.show();
    }
}
```

أول تطبيق JAVAFX

الخطوة 7: إنشاء حدث للزر

- حيث يقوم تطبيقنا بطباعة "مرحبا بالعالم" لحدث على الزر.
- نحتاج إلى إنشاء حدث للزر. لهذا الغرض، استدعِ الدالة `setOnAction()` على الزر، وعرّف معالج أحداث فئة مجهول كمعامل للطريقة.

```
btn1.setOnAction(new EventHandler<ActionEvent>() {

    @Override
    public void handle(ActionEvent arg0) {
        // TODO Auto-generated method stub
        System.out.println( x: "hello world");
    }

});
```

زر في اللوحة

فئة الألوان

يحدد JavaFX فئة Paint المجردة لرسم العقدة.

، `javafx.scene.paint.Color` هي فئة فرعية محددة من `Paint`

والتي تستخدم لتغليف الألوان.

فئة الألوان

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.paint.Color

-red: double
-green: double
-blue: double
-opacity: double

+Color(r: double, g: double, b: double, opacity: double)
+brighter(): Color
+darker(): Color
+color(r: double, g: double, b: double): Color
+color(r: double, g: double, b: double, opacity: double): Color
+rgb(r: int, g: int, b: int): Color
+rgb(r: int, g: int, b: int, opacity: double): Color

The red value of this Color (between 0.0 and 1.0).

The green value of this Color (between 0.0 and 1.0).

The blue value of this Color (between 0.0 and 1.0).

The opacity of this Color (between 0.0 and 1.0).

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color that is a brighter version of this Color.

Creates a Color that is a darker version of this Color.

Creates an opaque Color with the specified red, green, and blue values.

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255 and a given opacity.

فئة الألوان

• يمكن إنشاء مثل اللون باستخدام المنشئ التالي:

• لون عام (r مزدوج، g مزدوج، b مزدوج، تعقيم مزدوج)؛

التي يحدد b و r اللون من خلال مكوناته الحمراء والخضراء والزرقاء، g يقيم تتراوح من 0.0 (الظل الأعماق) إلى 1.0 (الظل

• تحدد قيمة العتامة شفافية اللون داخل

النطاق من 0.0 (شفاف تمامًا) إلى 1.0 (غير شفاف تمامًا).

يُعرف هذا بنموذج RGBA، حيث يرمز RGBA إلى الأحمر والأخضر والأزرق وألفا. تشير قيمة ألفا إلى التعقيم.

فئة الألوان

• فئة اللون غير قابلة للتغيير. بمجرد تغيير كائن اللون،

تم إنشاؤه، لا يمكن تغيير خصائصه.

تعيد الطريقة لونًا جديدًا بدرجات أكبر من الأحمر والأخضر

القيم الزرقاء، وتعيد طريقة `darker()` لونًا جديدًا

مع قيم أصغر للألوان الأحمر والأخضر والأزرق.

فئة الخط

• يتم تعريف الخط من خلال اسمه ووزنه وموقعه وحجمه.

Arial و Courier و Times New Roman • هي أمثلة لأسماء الخطوط.

• يمكنك الحصول على قائمة بأسماء عائلات الخطوط المتاحة من خلال استدعاء دالة `getFontNames()` الثابتة. تُرجع هذه الدالة `List<String>`.

• هناك وضعان للخط يتم تعريفهما كثوابت في

فئة: `FontPosture`

- `FontPosture.ITALIC`
- `FontPosture.REGULAR`
- كائن الخط ثابت. بمجرد إنشائه، لا يمكن تغيير خصائصه

فئة الخط

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.text.Font

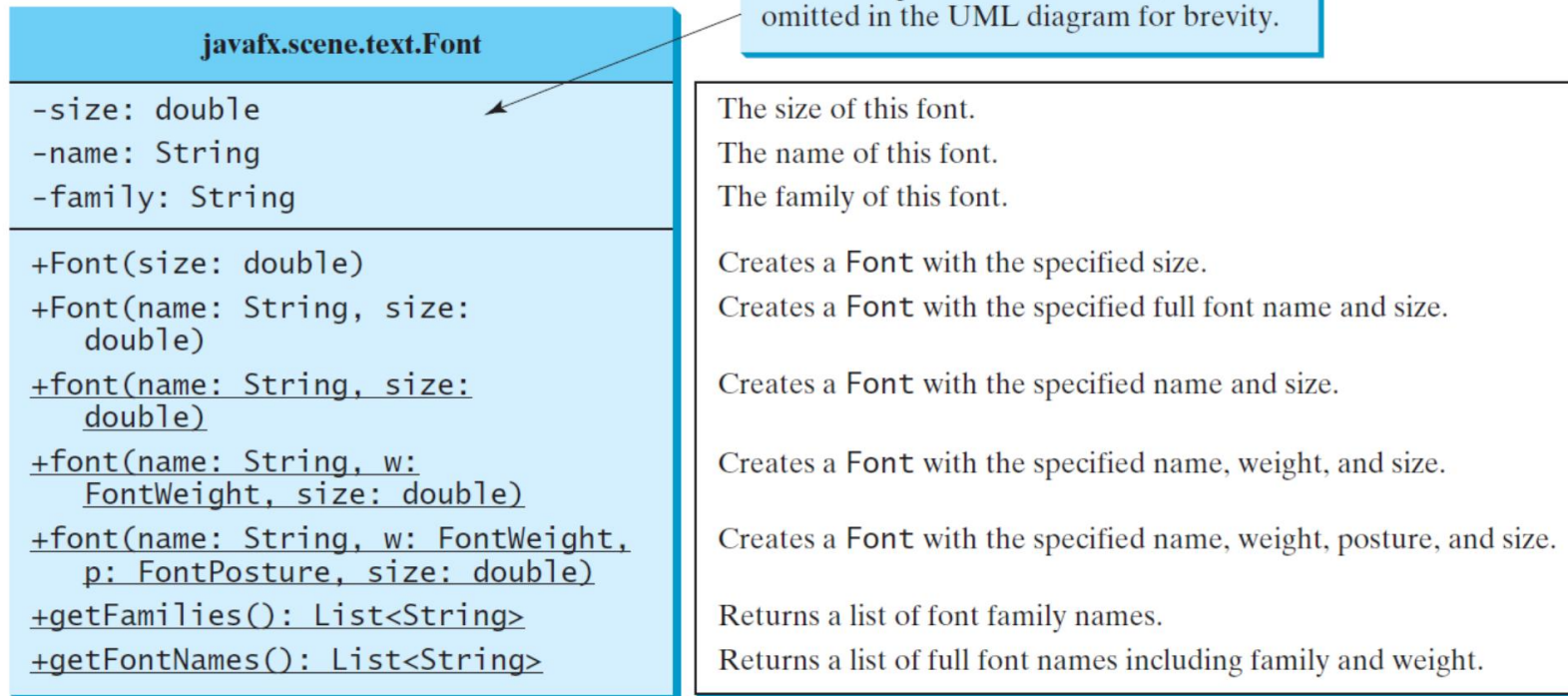
-size: double
-name: String
-family: String

+Font(size: double)
+Font(name: String, size: double)
+font(name: String, size: double)
+font(name: String, w: FontWeight, size: double)
+font(name: String, w: FontWeight, p: FontPosture, size: double)
+getFamilies(): List<String>
+getFontNames(): List<String>

The size of this font.
The name of this font.
The family of this font.

Creates a Font with the specified size.
Creates a Font with the specified full font name and size.
Creates a Font with the specified name and size.
Creates a Font with the specified name, weight, and size.
Creates a Font with the specified name, weight, posture, and size.
Returns a list of font family names.
Returns a list of full font names including family and weight.

فئة الخط



الصورة وعرض الصورة

• تمثل فئة `javafx.scene.image.Image` صورة رسومية وتُستخدم لتحميل صورة من اسم ملف محدد أو عنوان URL.

• على سبيل المثال،

صورة جديدة ("image/us.gif")

ينشئ كائن صورة لملف الصورة `us.gif` ضمن الدليل `image` في دليل فئة `Java`

9

صورة جديدة ("http://liveexample.pearsoncmg.com/book/image/us.gif")

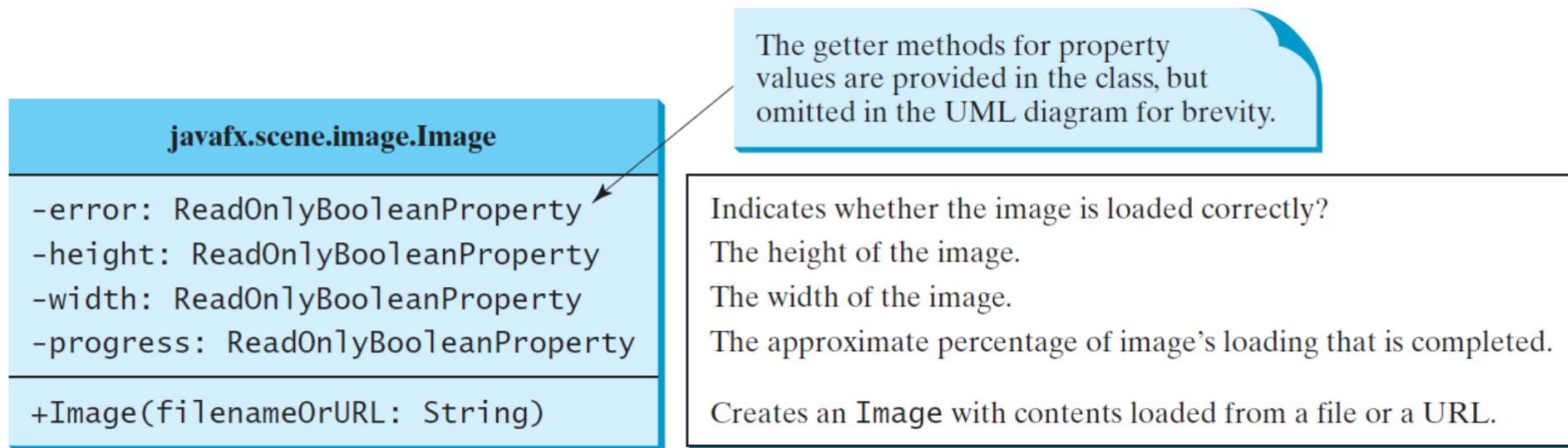
ينشئ كائن صورة لملف الصورة في عنوان URL على الويب.

الصورة وعرض الصورة

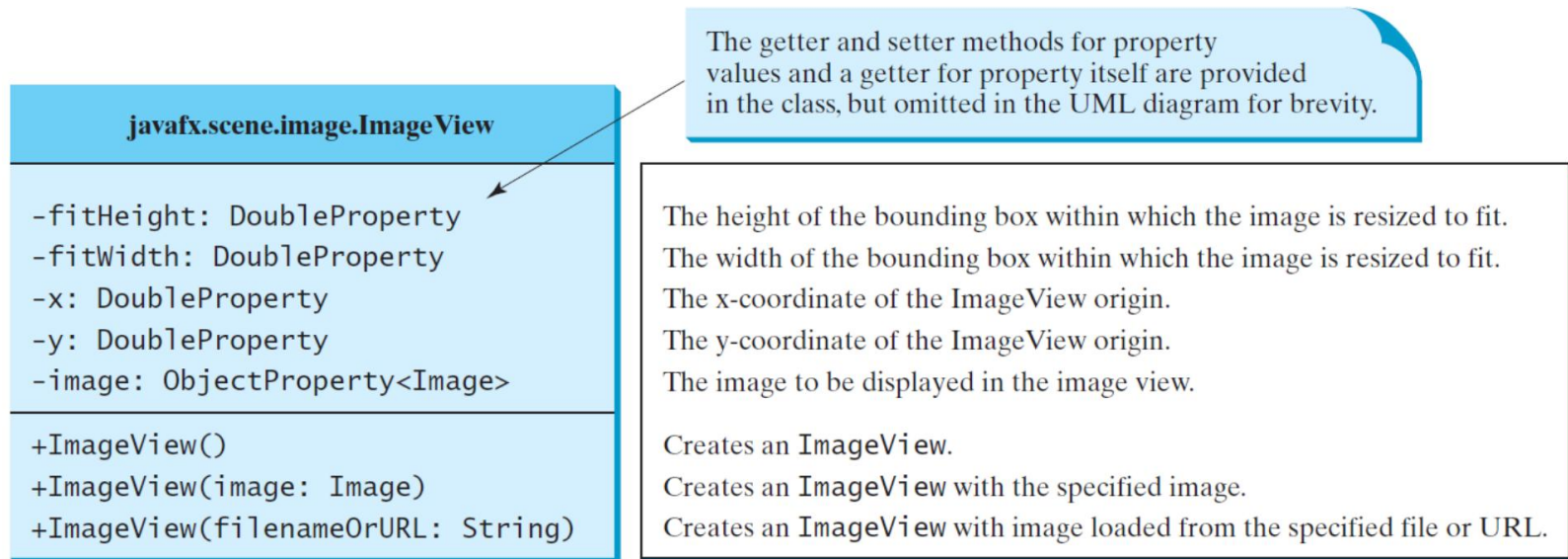
`javafx.scene.image.ImageView` هي عقدة لعرض الصور. يمكن إنشاء `ImageView` من كائن صورة.

ملاحظة : يمكن مشاركة كائن صورة بين عدة عُقد. في هذه الحالة، تتم مشاركته بين ثلاث عُقد `ImageView`. مع ذلك، لا يمكن مشاركة عُقدة مثل `ImageView` لا يمكنك وضع `ImageView` عدة مرات في جزء أو مشهد.

فئة الصورة



فئة عرض الصور



عرض الصورة

أي سؤال؟